

Assignment 6.4

AI Assisted Coding

Name: Navya Revuri

HTNO: 2303A52483

Batch 44

Task 1:

Prompt:

Student Performance Evaluation System

class Student:

```
#def __init__(self, name, roll_number, marks):
```

```
    self.name = name
```

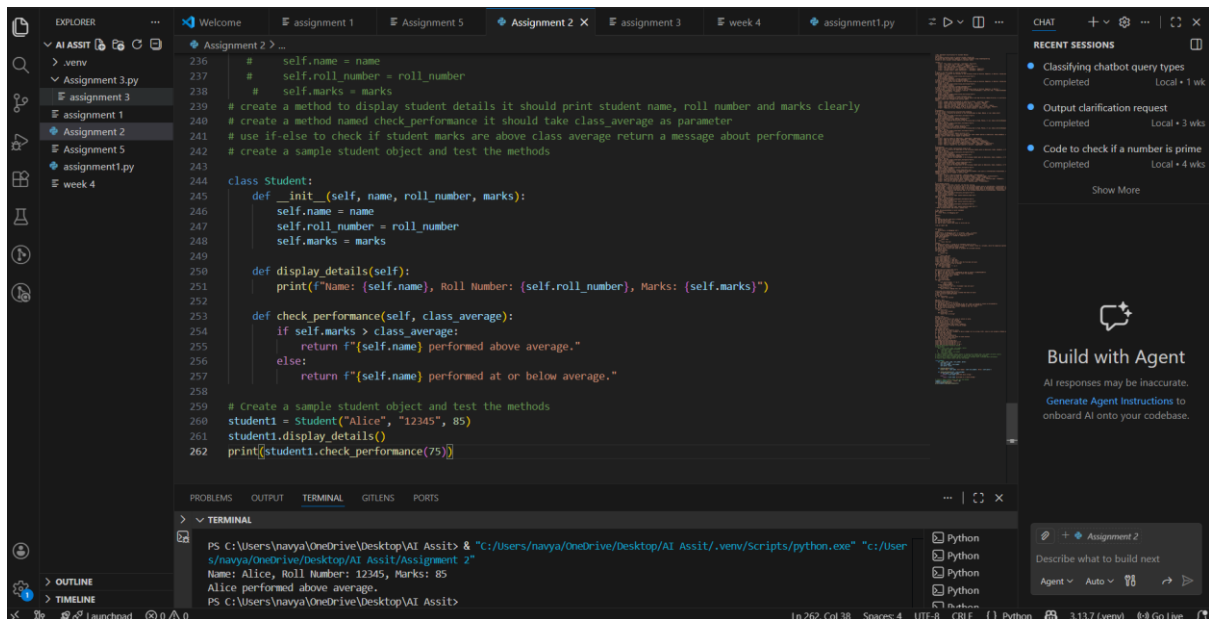
```
    self.roll_number = roll_number
```

```
    self.marks = marks
```

```
# Create a method to display student details it should print student name, roll number and marks clearly. Create a method named check_performance it should take class_average as parameter.
```

```
#use if-else to check if student marks are above class average return a message about performance
```

```
#Create a sample student object and test the methods
```



Observation – Student Performance Evaluation System

In this task, GitHub Copilot was used to complete the implementation of a Python class based on comment-based prompts. After providing the class skeleton and descriptive comments, Copilot successfully generated the required methods.

The AI assistant created:

- A `display_details()` method that prints student information using class attributes (`self.name`, `self.roll_number`, `self.marks`).
- A `check_performance()` method that uses an **if-else conditional statement** to compare student marks with the class average and return an appropriate message.

Task 2:

Prompt:

list of sensor readings

readings = [10, 15, 22, 9, 8, 30]

Write a for loop to iterate through the readings identify even numbers using modulus operator Calculate square of even numbers. Print the number and its square in a readable format

```
262 print(student1.check_performance(75))"""
263 # list of sensor readings
264 readings = [10, 15, 22, 9, 8, 30]
265 # write a for loop to iterate through the readings
266 # identify even numbers using modulus operator
267 # calculate square of even numbers
268 # print the number and its square in a readable format
269 for reading in readings:
270     if reading % 2 == 0:
271         square = reading ** 2
272         print(f"Number: {reading}, Square: {square}")
273
```

PROBLEMS OUTPUT TERMINAL GITLENS PORTS

TERMINAL

```
s:\navya\OneDrive\Desktop\AI Assit\Assignment 2"
Number: 10, Square: 100
Number: 22, Square: 484
Number: 8, Square: 64
Number: 30, Square: 900
PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Python Python Python Python Python Python Python Python Python Python

Build with Agent

AI responses may be inaccurate.

Generate Agent Instructions to onboard AI onto your codebase.

Assignment 2

Describe what to build next

Agent Auto 78

Ln 264, Col 1 Spaces: 4 UTF-8 CRLF Python 3.13.7 (x64) 8:4 Go Live

Observation

Using the provided comment prompts, GitHub Copilot generated the remaining loop logic successfully. The AI created a proper **for loop**, used the **modulus operator (%)** to identify even numbers, and calculated the square of each valid reading. The output was printed in a clear and readable format, and the program executed without errors. This task shows how AI can assist in writing loops and conditional statements efficiently while still requiring user verification.

Task 3:

Prompt:

Banking Transaction Simulation

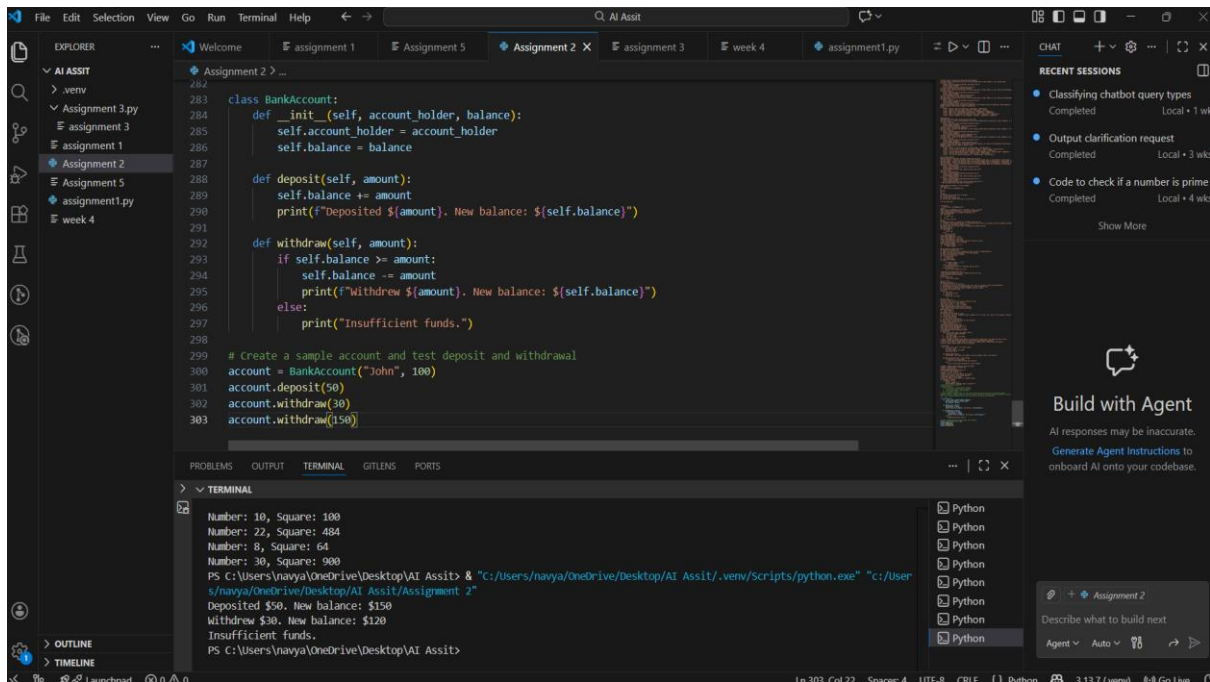
class Bank Account:

```
def __init__(self, account_holder, balance):
```

```
    self.account_holder = account_holder
```

```
    self.balance = balance
```

Create a method to deposit money into the account update the balance and print confirmation message. Create a method to withdraw money. Use if-else to check if balance is sufficient. Prevent withdrawal if balance is insufficient. Print user friendly messages. Create a sample account and test deposit and withdrawal



```
class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited ${amount}. New balance: ${self.balance}")

    def withdraw(self, amount):
        if self.balance >= amount:
            self.balance -= amount
            print(f"Withdrew ${amount}. New balance: ${self.balance}")
        else:
            print("Insufficient funds.")

# Create a sample account and test deposit and withdrawal
account = BankAccount("John", 100)
account.deposit(50)
account.withdraw(30)
account.withdraw(150)
```

```
Number: 10, Square: 100
Number: 22, Square: 484
Number: 8, Square: 64
Number: 30, Square: 900
PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/python.exe" "C:/Users/navya/OneDrive/Desktop/AI Assit/Assignment 2"
Deposited $50. New balance: $150
Withdrew $30. New balance: $120
Insufficient funds.
PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

GitHub Copilot successfully generated the deposit and withdraw methods based on the comment prompts. The AI used class attributes through self, applied **if-else conditional logic** to prevent withdrawals when the balance was insufficient, and produced clear user-friendly messages. The program executed correctly and demonstrated how AI can help build functional class methods while still requiring human review.

Task 4:

Prompt:

list of students with name and score

```
students = [{"name": "Asha", "score": 80},
```

```
            {"name": "Ravi", "score": 65},
```

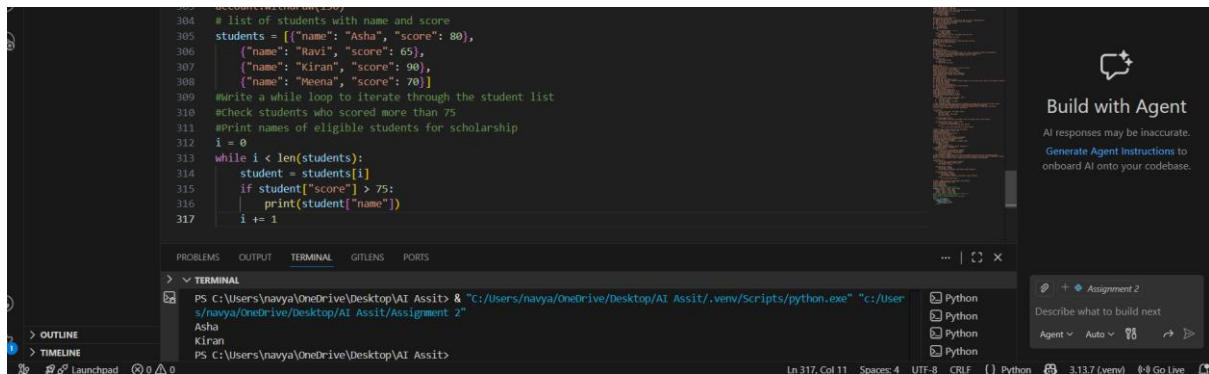
```
            {"name": "Kiran", "score": 90},
```

```
            {"name": "Meena", "score": 70}]
```

#Write a while loop to iterate through the student list

#Check students who scored more than 75

#Print names of eligible students for scholarship



The screenshot shows a code editor with a Python script on the left and a terminal window on the right. The script defines a list of students and uses a while loop to print the names of students who scored more than 75. The terminal output shows the execution of the script, which prints the names 'Asha', 'Kiran', and 'Meena'.

```
304 # list of students with name and score
305 students = [{"name": "Asha", "score": 80},
306             {"name": "Ravi", "score": 65},
307             {"name": "Kiran", "score": 90},
308             {"name": "Meena", "score": 70}]
309 #Write a while loop to iterate through the student list
310 #check students who scored more than 75
311 #Print names of eligible students for scholarship
312 i = 0
313 while i < len(students):
314     student = students[i]
315     if student["score"] > 75:
316         print(student["name"])
317     i += 1
```

Terminal Output:

```
PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/python.exe" "C:/User
s/navya/OneDrive/Desktop/AI Assit/Assignment 2"
Asha
Kiran
Meena
PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation– Scholarship Eligibility (While Loop)

GitHub Copilot generated the while loop based on the given prompts. The loop correctly iterated through the list using an index, applied a conditional check to identify students scoring above 75, and printed the eligible names. The program executed successfully and demonstrated how AI can assist in generating loop and condition-based logic

Task 5:

Prompt:

Online Shopping Cart Module

class Shopping Cart:

def __init__(self):

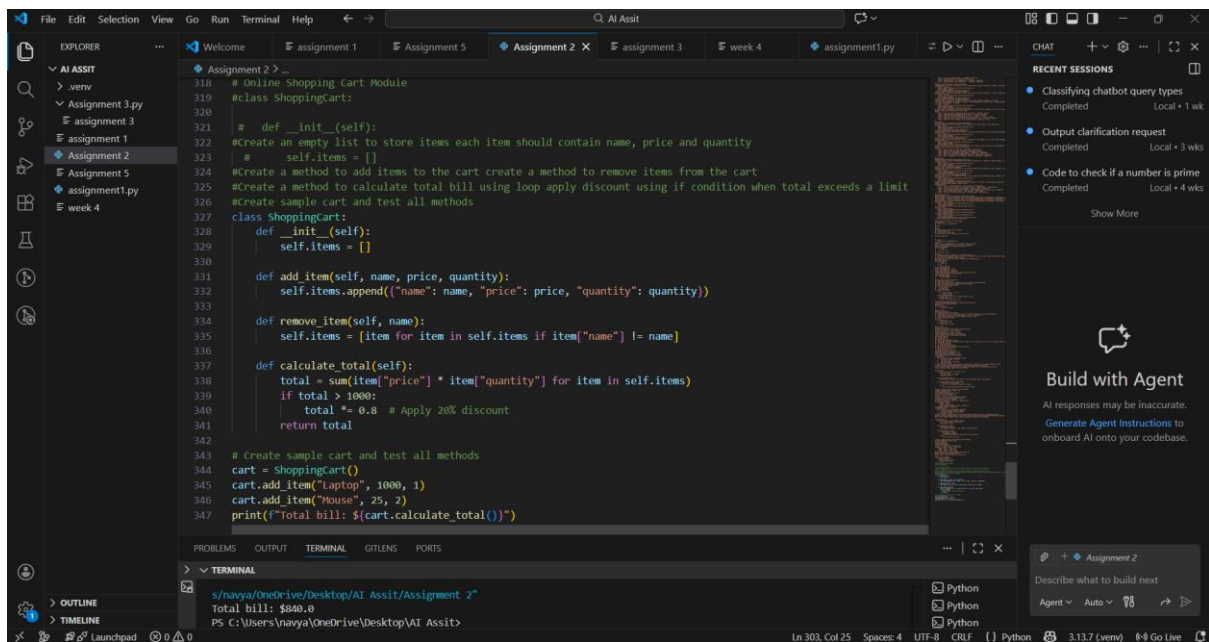
#Create an empty list to store items each item should contain name, price and quantity

self.items= []

#Create a method to add items to the cart create a method to remove items from the cart

#Create a method to calculate total bill using loop apply discount using if condition when total exceeds a limit

#Create sample cart and test all methods



Observation – Online Shopping Cart Module

Using the comment prompts, GitHub Copilot completed the Shopping Cart class with methods to add items, remove items, and calculate the total bill. The AI correctly used loops and conditional statements to compute the total and apply a discount when the amount exceeded a limit. The program ran successfully and showed how AI can help build functional class-based applications efficiently.